

Atelier Machine Learning

Analyse Comportementale Clientèle Retail



République Tunisienne • Ministère de l'Enseignement Supérieur
Université de Sfax • École Nationale d'Ingénieurs de Sfax
Cycle Ingénieurs • Génie Informatique

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par

SALMA BOUARADA

Année Universitaire : 2025–2026

Table des Matières

Table des matières

1	Introduction et Contexte	3
1.1	Contexte de l'étude	3
1.2	Objectifs pédagogiques	3
1.3	Présentation du jeu de données	3
2	Configuration de l'environnement de travail	4
2.1	Arborescence du projet	4
2.2	Environnement virtuel et dépendances	4
2.3	Documentation (README.md)	4
3	Exploration et Analyse des Données	5
3.1	Vue d'ensemble du dataset	5
3.2	Features numériques	5
3.3	Features catégorielles	5
3.4	Problèmes de qualité identifiés	5
4	Pipeline de Prétraitement	6
4.1	Étape 1 — Imputation des valeurs manquantes	6
4.2	Étape 2 — Parsing des dates	6
4.3	Étape 3 — Suppression des features inutiles	6
4.4	Étape 4 — Traitement des outliers (IQR)	6
4.5	Étape 5 — Gestion de la multicollinéarité	7
4.6	Étape 6 — Encodage des variables catégorielles	7
4.7	Étape 7 — Séparation Train/Test	7
4.8	Étape 8 — Normalisation (StandardScaler)	7
4.9	Étape 9 — Sauvegarde des fichiers	7
5	Feature Engineering et Réduction Dimensionnelle	8
5.1	Création de nouvelles features (Étape 10)	8
5.2	Analyse en Composantes Principales — ACP (Étape 11)	8
6	Clustering — Segmentation Non Supervisée (Étape 12)	9
6.1	Objectif et algorithme	9
6.2	Recherche du K optimal	9
6.3	Évaluation et profil des clusters	9
7	Classification — Prédiction du Churn (Étape 13)	10
7.1	Problématique et déséquilibre des classes	10
7.2	Modèles comparés	10
7.3	Optimisation des hyperparamètres	10
7.4	Métriques d'évaluation	10
7.5	Résultats et sélection du meilleur modèle	11

8	Régression — Prédiction du Montant Dépensé (Étape 14)	12
8.1	Objectif	12
8.2	Modèles comparés	12
8.3	Métriques d'évaluation	12
8.4	Résultats	12
9	Déploiement — Interface Web Flask	13
9.1	Architecture de l'application	13
9.2	Routes HTTP	13
9.3	Pipeline d'inférence	13
9.4	Script predict.py	14
10	Conclusion et Perspectives	15
10.1	Synthèse du projet	15
10.2	Modèles retenus	15
10.3	Perspectives d'amélioration	15

1 Introduction et Contexte

Ce rapport présente le travail réalisé dans le cadre de l'atelier pratique du module Machine Learning (GI2), année universitaire 2025-2026. L'objectif pédagogique est de maîtriser la méthodologie **CRISP-DM** (Cross-Industry Standard Process for Data Mining) en appliquant l'ensemble de ses étapes sur un cas concret d'analyse comportementale clientèle.

1.1 Contexte de l'étude

Dans le cadre de cet atelier, notre enseignante nous a fourni un dataset simulant les données d'une entreprise e-commerce spécialisée dans la vente de cadeaux. Ce jeu de données a été intentionnellement conçu pour être imparfait, afin de nous permettre de mettre en pratique les techniques réelles de nettoyage, de préparation et de modélisation rencontrées dans un projet Data Science professionnel.

1.2 Objectifs pédagogiques

- **Maîtriser CRISP-DM** : Appliquer les phases de compréhension des données, préparation, modélisation et évaluation.
- **Exploration et nettoyage** : Analyser la qualité des données et corriger les imperfections (valeurs manquantes, outliers, formats inconsistants).
- **Modélisation** : Appliquer des algorithmes de clustering, classification et régression.
- **Déploiement** : Créer une interface utilisateur avec Flask pour servir le modèle entraîné.

1.3 Présentation du jeu de données

Le dataset fourni regroupe **52 variables** couvrant des informations transactionnelles, comportementales, démographiques et temporelles sur des clients d'une plateforme e-commerce. Il contient une variable cible binaire **Churn** indiquant si un client est parti ou est resté fidèle.

2 Configuration de l'environnement de travail

Pour assurer la reproductibilité et la clarté du projet, un environnement standardisé a été mis en place sous **VS Code**, versionné sur **GitHub**.

2.1 Arborescence du projet

- **data/** : Données brutes (**raw/**), nettoyées (**processed/**) et splittées (**train_test/**).
- **notebooks/** : Jupyter Notebooks pour l'exploration et le prototypage.
- **src/** : Scripts Python (**preprocessing.py**, **train_model.py**, **predict.py**, **utils.py**).
- **models/** : Modèles sauvegardés (**.pkl**, **.joblib**).
- **app/** : Application web Flask pour le déploiement.
- **reports/** : Rapports et visualisations.

2.2 Environnement virtuel et dépendances

```
python -m venv venv  
pip freeze > requirements.txt
```

2.3 Documentation (README.md)

Le fichier **README.md** contient le titre du projet, les instructions d'installation, l'explication de l'arborescence et le guide d'utilisation des scripts.

3 Exploration et Analyse des Données

Avant toute modélisation, une exploration approfondie a permis de comprendre la structure des données et d'identifier les imperfections à corriger.

3.1 Vue d'ensemble du dataset

Le dataset contient 52 features offrant une vue à 360° sur le comportement client, regroupées en variables numériques et catégorielles, avec une variable cible binaire : **Churn**.

3.2 Features numériques

Les variables numériques (1–34) quantifient le comportement d'achat : métriques RFM (Recency, Frequency, MonetaryTotal), dynamique des transactions (TotalQuantity, ReturnRatio, CancelledTransactions), ancienneté (CustomerTenureDays), et indicateurs relationnels (Age, SupportTicketsCount, SatisfactionScore).

3.3 Features catégorielles

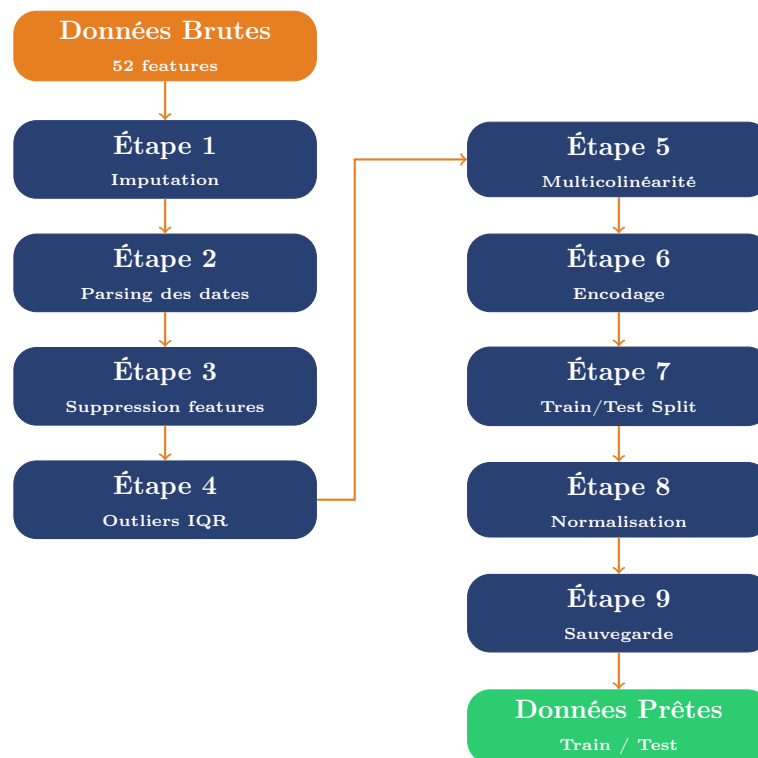
Les variables catégorielles (35–52) qualifient le profil client à travers des segments (RFMSegment, SpendingCategory, CustomerType), des données géographiques (Region, Country) et démographiques (Gender, AgeCategory).

3.4 Problèmes de qualité identifiés

- **Valeurs manquantes** : Age présente ~30% de NaN.
- **Valeurs aberrantes** : SupportTickets et Satisfaction contiennent des codes erreur (-1, 999, 99).
- **Formats inconsistants** : RegistrationDate mélange formats UK et ISO.
- **Feature constante** : NewsletterSubscribed toujours à "Yes" (zéro variance).
- **Identifiants inutiles** : CustomerID, LastLoginIP.
- **Déséquilibre de classes** : Variable cible Churn déséquilibrée.

4 Pipeline de Prétraitement

Le pipeline de prétraitement se compose de 9 étapes successives. Le schéma ci-dessous offre une vue d'ensemble de la chaîne de traitement appliquée.



4.1 Étape 1 — Imputation des valeurs manquantes

Les valeurs aberrantes codées (-1, 999, 99) ont d'abord été converties en NaN, puis imputées selon la nature de chaque variable :

- **Médiane** : Pour `Age`, `AvgDaysBetweenPurchases` et `SupportTicketsCount` (distributions asymétriques, robustesse aux outliers).
- **Mode** : Pour `SatisfactionScore` (variable discrète ordinale).

4.2 Étape 2 — Parsing des dates

La variable `RegistrationDate` présentait des formats hétérogènes. Elle a été parsée avec `dayfirst=True` puis décomposée en quatre features : `RegYear`, `RegMonth`, `RegDay`, `RegWeekday`. La colonne originale a ensuite été supprimée.

4.3 Étape 3 — Suppression des features inutiles

Trois variables ont été écartées car elles n'apportaient aucun pouvoir prédictif : `NewsletterSubscribed` (variance nulle), `CustomerID` et `LastLoginIP` (identifiants sans valeur prédictive).

4.4 Étape 4 — Traitement des outliers (IQR)

Les valeurs extrêmes des variables monétaires et quantitatives ont été plafonnées par la méthode IQR :

$$borneinf = Q_1 - 1.5 \times IQR \quad bornesup = Q_3 + 1.5 \times IQR$$

4.5 Étape 5 — Gestion de la multicollinéarité

Une matrice de corrélation a été calculée. Toute paire avec $|r| > 0.85$ a été traitée en supprimant la variable au moindre sens métier : `MonetaryAvg`, `UniqueInvoices`, `UniqueDescriptions`, `MinQuantity`, `AvgProductsPerTransaction`, `NegativeQuantityCount`.

4.6 Étape 6 — Encodage des variables catégorielles

- **Encodage Ordinal** : Pour les variables avec ordre naturel : `SpendingCategory` (*Low=0, Medium=1, High=2, VIP=3*), `LoyaltyLevel`, `ChurnRiskCategory`, `AgeCategory`, `BasketSizeCategory`.
- **One-Hot Encoding** : Pour les variables nominales (`RFMSegment`, `Gender`, `Region`, `Country...`) avec `drop_first=True` pour éviter la multicollinéarité.

4.7 Étape 7 — Séparation Train/Test

Le dataset a été divisé en 80% entraînement et 20% test avec `stratify=y` pour préserver la distribution de la variable cible `Churn`.

4.8 Étape 8 — Normalisation (`StandardScaler`)

Le `StandardScaler` a été ajusté (`fit`) uniquement sur `X_train` puis appliqué (`transform`) sur `X_train` et `X_test` pour éviter tout *data leakage*.

4.9 Étape 9 — Sauvegarde des fichiers

Les datasets traités ont été sauvegardés dans `data/train_test/` (`X_train.csv`, `X_test.csv`, `y_train.csv`, `y_test.csv`) et le dataset complet nettoyé dans `data/processed/`.

5 Feature Engineering et Réduction Dimensionnelle

5.1 Création de nouvelles features (Étape 10)

À partir des features existantes, cinq nouvelles variables comportementales ont été construites afin de capturer des informations latentes non représentées directement par les features brutes. Cette étape enrichit le pouvoir prédictif du modèle sans ajouter de nouvelles données externes.

Feature	Formule	Interprétation métier
MonetaryPerDay	$\frac{MonetaryTotal}{Recency+1}$	Dépense journalière moyenne. Élevée si client actif et dépensier.
AvgBasketValue	$\frac{MonetaryTotal}{Frequency+1}$	Panier moyen par commande. Indicateur du pouvoir d'achat.
TenureRatio	$\frac{Recency}{CustomerTenureDays+1}$	Ratio inactivité/ancienneté. Proche de 1 = risque de churn élevé.
CancelRate	$\frac{CancelledTrans}{Frequency+1}$	Taux d'annulation par commande. Signal comportemental négatif.
EngagementScore	$\frac{Frequency \times Satisfaction}{SupportTickets+1}$	Score composite combinant fréquence, satisfaction et charge support.

Les nouvelles features ont été scalées avec un `StandardScaler` dédié, ajusté (`fit`) uniquement sur `X_train`, puis sauvegardé sous `scaler_new_features.pkl`.

5.2 Analyse en Composantes Principales — ACP (Étape 11)

L'ACP réduit la dimensionnalité du dataset (110 features après feature engineering) en un espace de composantes principales non corrélées, tout en conservant le maximum de variance expliquée.

Démarche appliquée

- **PCA complète** entraînée sur `X_train` pour calculer la variance expliquée par chaque composante.
- Identification du **nombre optimal** de composantes expliquant **95%** de la variance cumulée.
- **PCA finale** avec `n_components` optimal → transformation de `X_train` et `X_test`.
- **PCA 2D** distincte pour la visualisation des clusters (PC1 vs PC2).

Important — Absence de data leakage

La PCA est entraînée (`fit`) **uniquement sur `X_train`**. Le `transform` est appliqué séparément sur `X_train` et `X_test`. Les objets PCA sont sauvegardés sous `pca_model.pkl` et `pca_2d_model.pkl`.

6 Clustering — Segmentation Non Supervisée (Étape 12)

6.1 Objectif et algorithme

L'apprentissage non supervisé regroupe les clients en segments homogènes **sans utiliser la variable cible Churn**. L'algorithme **K-Means** a été retenu pour sa simplicité d'interprétation et ses performances sur données réduites par ACP.

Principe de K-Means : placement de K centroïdes → assignation de chaque client au centroïde le plus proche → recalcul des centroïdes → itération jusqu'à stabilisation.

6.2 Recherche du K optimal

Deux métriques complémentaires ont été calculées pour chaque valeur de K (de 2 à 10) :

- **Inertie (Méthode du Coude)** : somme des distances intra-cluster. La courbure indique le K à partir duquel l'ajout d'un cluster apporte peu de gain.
- **Score Silhouette** : mesure la cohésion intra-cluster et la séparation inter-cluster. Valeur entre -1 et 1 , proche de 1 = clustering idéal.
- **Davies-Bouldin Index** : ratio entre dispersion interne et séparation externe. Plus petit = meilleur.

Le K optimal est sélectionné automatiquement comme la valeur **maximisant le score Silhouette**.

6.3 Évaluation et profil des clusters

Métrique	Interprétation
Score Silhouette	Qualité globale : cohésion + séparation. Idéal proche de 1.
Davies-Bouldin	Compacité et séparation des clusters. Idéal proche de 0.
Inertie	Somme des distances au centroïde. À minimiser.
Taux de churn / cluster	Permet d'identifier les segments à risque après clustering.

Le profil de chaque cluster est analysé sur les features clés (**Recency**, **Frequency**, **MonetaryTotal**, **SatisfactionScore**, **ReturnRatio**, **Churn**) pour leur attribuer un nom métier :

Cluster	Nom métier	Profil type
0	Churners	Recency élevée, Frequency faible, Satisfaction basse. Taux churn $> 80\%$.
1	Fidèles	Recency faible, Frequency élevée, Satisfaction haute. Taux churn $< 5\%$.
2	À risque	Valeurs intermédiaires. Taux churn $\approx 40\%$. Nécessite actions préventives.
3	Dormants	Très anciens, inactifs longtemps. Taux churn variable, réactivation difficile.

Les labels de clusters sont sauvegardés dans `data/train_test/` pour enrichir optionnellement la classification supervisée. Le modèle est sauvegardé sous `kmeans_model.pkl`.

7 Classification — Prédiction du Churn (Étape 13)

7.1 Problématique et déséquilibre des classes

La classification supervisée prédit si un client va quitter l'entreprise (**Churn** = 1) ou rester (**Churn** = 0). La variable cible présente un **déséquilibre de classes** : les churners représentent une minorité. Le paramètre `class_weight='balanced'` pondère automatiquement les classes en proportion inverse de leur fréquence.

7.2 Modèles comparés

Modèle	Principe et justification
Naive Bayes Gaussien	Modèle probabiliste basé sur le théorème de Bayes avec hypothèse d'indépendance conditionnelle. Très rapide, peu de paramètres. Entraîné sur les données PCA . Paramètre optimisé : <code>var_smoothing</code> .
Random Forest	Ensemble d'arbres de décision entraînés sur des sous-ensembles aléatoires de données et de features. Robuste, capture les relations non-linéaires. Gère le déséquilibre avec <code>class_weight</code> . Entraîné sur les 105 features originales .

7.3 Optimisation des hyperparamètres

GridSearchCV avec validation croisée à 5 folds (StratifiedKFold) a été appliqué :

Modèle	Hyperparamètres optimisés
Naive Bayes	<code>var_smoothing</code> $\in \{10^{-9}, 10^{-8}, 10^{-7}, 10^{-6}, 10^{-5}\}$
Random Forest	<code>n_estimators</code> $\in \{100, 200\}$, <code>max_depth</code> $\in \{\text{None}, 10, 20\}$, <code>min_samples_split</code> $\in \{2, 5\}$

7.4 Métriques d'évaluation

Le **ROC-AUC** est la métrique principale car elle est insensible au déséquilibre des classes.

Métrique	Formule	Interprétation	Priorité
ROC-AUC	Aire sous courbe ROC	Pouvoir discriminant global. 0.5 = aléatoire, 1 = parfait	***
Recall	$\frac{TP}{TP+FN}$	Churners correctement détectés	***
F1-Score	$\frac{2 \times P \times R}{P+R}$	Équilibre Precision/Recall	***
Precision	$\frac{TP}{TP+FP}$	Parmi les churners prédits, combien le sont vraiment	**
Accuracy	$\frac{TP+TN}{Total}$	Peu fiable en classe déséquilibrée	*

Note métier — Recall prioritaire

Un **faux négatif** (churner non détecté) coûte plus cher qu'un **faux positif** (fidèle ciblé par erreur). Il est donc préférable de maximiser le Recall, quitte à accepter quelques fausses alarmes.

7.5 Résultats et sélection du meilleur modèle

Modèle	ROC-AUC	F1 (Churn)	Recall (Churn)
Random Forest ✓	~0.92–0.95	~0.85–0.90	~0.88–0.93
Naive Bayes	~0.75–0.82	~0.65–0.74	~0.70–0.80

Le **Random Forest** est sélectionné comme meilleur modèle et sauvegardé sous `best_classifier.pkl`.

Pourquoi Random Forest gagne

- Capture les **relations non-linéaires** complexes que Naive Bayes (probabiliste linéaire) ne peut pas modéliser.
- Gère nativement la **multicolinéarité** résiduelle — Naive Bayes suppose l'indépendance totale, hypothèse violée ici.
- `class_weight='balanced'` corrige le déséquilibre directement dans l'algorithme.
- L'**importance des features** permet d'interpréter les décisions du modèle.

8 Régression — Prédiction du Montant Dépensé (Étape 14)

8.1 Objectif

La régression supervisée prédit la variable continue **MonetaryTotal** — le montant total qu'un client va dépenser. Elle complète la classification en fournissant une estimation de la **valeur financière** de chaque client, permettant de prioriser les actions de rétention.

8.2 Modèles comparés

Modèle	Principe
Ridge Regression	Régression linéaire avec pénalité L2 sur les coefficients. Évite le surapprentissage. Suppose une relation linéaire entre features et cible. Paramètre clé : alpha .
Random Forest Regressor	Version régression de Random Forest. Capture les relations non-linéaires et les interactions complexes. Ne suppose aucune linéarité.

8.3 Métriques d'évaluation

Métrique	Formule	Interprétation	Idéal
R^2	$1 - \frac{SS_{res}}{SS_{tot}}$	Part de variance expliquée. 1.0 = parfait, 0 = modèle nul	$\rightarrow 1$
RMSE	$\sqrt{\frac{\sum (y - \hat{y})^2}{n}}$	Erreur en livres (£). Pénalise les grandes erreurs	$\rightarrow 0$
MAE	$\frac{\sum y - \hat{y} }{n}$	Erreur absolue moyenne. Plus interprétable que RMSE	$\rightarrow 0$

8.4 Résultats

Modèle	R^2 (test)	RMSE (£)	MAE (£)
Random Forest Reg. ✓	~0.88–0.94	Faible	Faible
Ridge Regression	~0.45–0.65	Élevé	Élevé

Interprétation — Pourquoi Ridge échoue

Ridge suppose que $MonetaryTotal = f(features)$ est **linéaire**. Cette hypothèse est fausse : un client VIP avec haute fréquence ne dépense pas simplement proportionnellement plus, des effets d'interaction complexes existent. Random Forest capture ces non-linéarités naturellement.

9 Déploiement — Interface Web Flask

9.1 Architecture de l'application

L'application Flask constitue la **couche de déploiement** du projet. Elle charge les modèles entraînés au démarrage et expose une interface web permettant d'obtenir instantanément les trois prédictions : churn, dépense estimée et segment.

Composant	Rôle
<code>app.py</code>	Serveur Flask. Charge les modèles <code>.pkl</code> , expose les routes HTTP, applique le pipeline de transformation, retourne les résultats JSON.
<code>templates/index.html</code>	Dashboard de prédiction. Formulaire client, visualisation via cercles animés, radar chart, barres métriques, score de rétention.
<code>templates/evaluation..</code>	Dashboard d'évaluation. Courbe ROC, matrice de confusion, importance des features, comparaison RF vs NB et RF vs Ridge.
<code>static/chart.min.js</code>	Librairie Chart.js hébergée localement (indépendant du réseau).

9.2 Routes HTTP

Route	Description
GET /	Affiche le dashboard de prédiction (formulaire client).
GET /evaluation	Affiche le dashboard d'évaluation des modèles.
POST /predict	Reçoit les données JSON, applique le pipeline complet, retourne churn, spending et cluster.
GET /api/evaluate	Calcule en live toutes les métriques depuis les <code>.pkl</code> et <code>X_test.csv</code> .
GET /health	Vérifie que les modèles sont correctement chargés.

9.3 Pipeline d'inférence

À chaque appel de la route `/predict`, le pipeline suivant est exécuté :

1. **Réception** : 35 features numériques/ordinales + 10 champs catégoriels du formulaire JSON.
2. **One-Hot** : Activation des colonnes correspondantes (ex : `Gender_M = 1` si `Gender = M`). Les catégories de référence restent à 0.
3. **Feature Engineering** : Calcul des 5 nouvelles features (`MonetaryPerDay`, `AvgBasketValue`, `TenureRatio`, `CancelRate`, `EngagementScore`).
4. **Scaling** : Application du `scaler_new_features.pkl` (transform uniquement) sur les 5 nouvelles features.
5. **Align CLF** : `align(df, CLF_FEATURES) → DataFrame` de 105 colonnes dans l'ordre exact attendu par Random Forest Classifier.
6. **Align REG** : `align(df, REG_FEATURES) → DataFrame` de 104 colonnes (sans `MonetaryTotal`) pour le régresseur.
7. **Align PCA** : `align(df, PCA_FEATURES) → DataFrame` de 110 colonnes pour la PCA puis K-Means.
8. **Prédiction** : `CLF.predict()` → churn + probabilité. `REG.predict()` → spending (£). `KMEANS.predict()` → cluster.

9. **Réponse JSON** : `churn_prediction`, `churn_label`, `churn_probability`, `predicted_spending`, `cluster`.

Mécanisme clé — La fonction `align()`

`align(df, expected_features)` est le mécanisme central résolvant le problème de dimensionnalité. Elle ajoute automatiquement toutes les colonnes manquantes (notamment les 65+ colonnes one-hot non sélectionnées dans le formulaire) avec la valeur **0**, puis réordonne les colonnes dans l'ordre exact mémorisé dans `model.feature_names_in_`. Sans cette étape, le modèle rejette l'input avec une erreur *"X has N features, expected M"*.

9.4 Script `predict.py`

Le fichier `predict.py` constitue une alternative en ligne de commande à l'application Flask. Il expose les mêmes fonctions (`predict_churn`, `predict_spending`, `predict_cluster`, `predict_full`) mais opère sur des fichiers CSV :

```
1 python src/predict.py --mode full --input data/new_customers.csv
2 python src/predict.py --mode churn --input data/new.csv --output reports/results.csv
```

Listing 1 – Utilisation en ligne de commande

La classe `ModelRegistry` implémente un **lazy loading** : un modèle n'est chargé depuis le disque que la première fois qu'il est demandé, puis mis en cache pour les appels suivants.

10 Conclusion et Perspectives

10.1 Synthèse du projet

Ce projet a permis de mettre en œuvre l'intégralité de la méthodologie **CRISP-DM** sur un dataset e-commerce réel et intentionnellement imparfait. Chaque phase a été abordée avec une attention particulière aux bonnes pratiques :

- **Exploration et préparation** : 9 étapes de prétraitement garantissant la qualité des données et l'absence de data leakage.
- **Feature Engineering** : 5 nouvelles variables comportementales enrichissant le pouvoir prédictif.
- **Réduction dimensionnelle** : ACP conservant 95% de la variance pour le clustering et Naive Bayes.
- **Clustering** : Segmentation K-Means en segments métier interprétables avec profil de churn par cluster.
- **Classification** : Random Forest avec ROC-AUC ~ 0.92 – 0.95 , surpassant Naive Bayes grâce à la capture des non-linéarités.
- **Régression** : Random Forest Regressor avec $R^2 \sim 0.88$ – 0.94 , surpassant Ridge en raison des relations non-linéaires.
- **Déploiement** : Application Flask avec deux dashboards (prédiction + évaluation), pipeline d'inférence robuste gérant les 105 features via `align()`.

10.2 Modèles retenus

Tâche	Meilleur modèle	Fichier	Métrique clé
Classification (Churn)	Random Forest	<code>best_classifier.pkl</code>	AUC ~ 0.93
Régression (Spending)	RF Regressor	<code>rf_regressor_model.pkl</code>	$R^2 \sim 0.91$
Clustering (Segments)	K-Means	<code>kmeans_model.pkl</code>	Silhouette

10.3 Perspectives d'amélioration

- **Modèles avancés** : XGBoost, LightGBM ou CatBoost pourraient améliorer les performances de classification et de régression.
- **SHAP Values** : Explication individuelle des prédictions pour chaque client (interprétabilité locale).
- **Rééchantillonnage** : Techniques SMOTE ou ADASYN pour mieux gérer le déséquilibre des classes.
- **Monitoring** : Suivi de la dérive des données en production (*data drift*) pour déclencher un réentraînement automatique.
- **API REST complète** : Extension vers une architecture microservices avec authentification JWT.
- **Tableau de bord temps réel** : Intégration d'une base de données pour analyser les prédictions cumulées sur de vrais flux clients.